SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO3 ASSESSMENT TOOL 2 – Code Review & Repository Evaluation

Course Code:	CSA10	**Course Title:**	Software Engineering
CO Assessed:	CO3	**Assessment:**	Assessment Tool 2 – Code Review & Repository Evaluation
Weightage:	20%	**Total Marks:**	25
CO3 Statement:	Build full-stack applications with an emphasis on containerization, version control, and collaborative development		
Student Name:		**Reg. No.:**	
Date:		**Duration:**	60 minutes

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Annexure A – Code Review & Repository Evaluation

Instructions to Students:

Each student has been assigned an individual problem statement. Based on the assigned problem statement, perform a **Code Review and Repository Evaluation** of your project. Analyze the quality of the source code, repository organization, documentation, coding practices, and overall maintainability. Provide appropriate screenshots, code snippets, diagrams, and justifications wherever applicable.
Assignment Questions

1. **Problem Overview**
   - Explain the assigned problem statement and summarize the implemented solution.
   - Describe the project objectives and key functionalities.
2. **Repository Organization**
   - Evaluate the repository structure, folder organization, and file naming conventions.
   - Justify how the repository layout supports maintainability and collaboration.
3. **Code Quality Review**
   - Review the source code for readability, modularity, coding standards, and documentation.
   - Identify strengths and areas for improvement.
4. **Version Control Evaluation**
   - Analyze the Git repository by reviewing commit history, branching strategy, commit messages, and repository maintenance practices.
   - Explain how version control supports collaborative software development.
5. **Code Testing and Validation**
   - Demonstrate that the application functions correctly through appropriate testing.
   - Present test cases, execution results, and screenshots as evidence.
6. **Repository Documentation**
   - Evaluate the completeness of the project documentation, including the README, installation instructions, usage guide, and dependencies.

    - Suggest improvements to enhance usability and maintainability.
7. **Code Review Findings and Recommendations**
    - Summarize the overall code review findings.
    - Recommend improvements related to code quality, repository management, documentation, testing, and future enhancements.

Expected Deliverables

- Code Review Report (10–15 pages)
- Git repository link
- Screenshots of repository structure and commit history
- Code review observations with examples
- Test execution screenshots
- Recommendations for improvement

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Analysis & Repository Organization(15 %)	Clearly explains the problem, solution, and repository structure with logical organization and justification.	Good explanation with minor omissions.	Basic explanation and repository organization.	Incomplete or poorly organized repository.
Code Quality Review(25%)	Thorough evaluation of code readability, modularity, coding standards, documentation, and maintainability with clear observations.	Good code review with minor gaps.	Basic review with limited analysis.	Superficial or inaccurate code review.
Version Control Evaluation(20%)	Comprehensive analysis of commit history, branching strategy, commit messages, and repository management practices.	Good evaluation with adequate explanation.	Basic evaluation with limited evidence.	Poor or incomplete version control analysis.
Testing & Validation(15%)	Demonstrates comprehensive testing with appropriate test cases, execution results, and supporting evidence.	Good testing with minor omissions.	Basic testing with limited evidence.	Inadequate or missing testing and validation.
Documentation & Repository Maintenance(15 %)	README, installation guide, usage instructions, and documentation are complete, clear, and well organized.	Documentation is mostly complete.	Basic documentation with missing details.	Poor or insufficient documentation.

Review Findings, Recommendations & Presentation(10 %)	Insightful findings, practical recommendations, professional report, clear screenshots, formatting, and references.	Good recommendations and presentation.	Basic recommendations with acceptable presentation.	Weak conclusions and poor presentation.

Criteria	**Mark**
System Requirement Analysis **(30%)**	/5
Selection of Software Architecture **(25%)**	/5
Architecture Diagram and Component Design **(20%)**	/5
Component Interaction and Quality Attribute Analysis **(15%)**	/5
Architectural Justification and Technical Presentation **(10%)**	/5
TOTAL	/25

PROBLEM STATEMENT - AI-Powered Digital Twin Platform for Smart City Infrastructure

Industry Context

Smart cities increasingly adopt digital twin technology to simulate and monitor urban infrastructure in real time. AI, IoT, GIS, and cloud computing enable virtual replicas of roads, utilities, buildings, and transportation systems for predictive maintenance and strategic planning.

Problem Statement

Urban infrastructure management is challenging due to aging assets, increasing population,and limited real-time visibility. Traditional monitoring systems provide fragmented information and lack predictive capabilities. An AI-powered digital twin platform should create virtual models of city infrastructure by integrating real-time IoT data. The system should simulateinfrastructure performance, detect anomalies, predict failures, and support data-driven urban planning. This solution will improve infrastructure reliability and maintenance efficiency.

Objectives

1. Create digital twins of city infrastructure.
2. Integrate real-time IoT data.
3. Predict infrastructure failures.
4. Monitor asset health continuously.
5. Support predictive maintenance.
6. Simulate urban scenarios.
7. Generate infrastructure performance reports.
8. Improve smart city planning.

Expected Outcomes

1. Enhanced infrastructure monitoring.
2. Reduced maintenance costs.
3. Improved asset reliability.
4. Better urban planning decisions.
5. Accurate predictive maintenance.
6. Increased operational efficiency.
7. Real-time infrastructure visualization.

1. PROBLEM OVERVIEW

1.1 Introduction

Smart cities increasingly use digital technologies to monitor and manage urban infrastructure. Roads, buildings, utilities, transportation systems, and other infrastructure assets generate large amounts of data through sensors and connected devices. However, traditional infrastructure monitoring systems often provide fragmented information and have limited predictive capabilities.

The proposed AI-Powered Digital Twin Platform for Smart City Infrastructure addresses this problem by creating virtual representations of physical infrastructure assets. The platform integrates IoT data, artificial intelligence, databases, backend services, and visualization technologies to continuously monitor infrastructure conditions.

A digital twin acts as a virtual representation of a physical asset. The system receives real-time or simulated sensor information and updates the corresponding digital model. AI-based analysis can then identify abnormal behavior, estimate potential failures, and support predictive maintenance.

1.2 Problem Statement

Urban infrastructure management is challenging because of aging infrastructure, increasing population, growing maintenance requirements, and limited real-time visibility.Traditional monitoring systems generally operate independently, resulting in fragmented information. They may identify existing problems but provide limited capability for predicting future failures.

The proposed system addresses these limitations by integrating real-time IoT data with digital twins and AI-based analytics. The platform monitors infrastructure continuously, detects anomalies, predicts potential failures, and provides information that can support maintenance and urban planning decisions.

1.3 Objectives

The major objectives of the proposed system are:

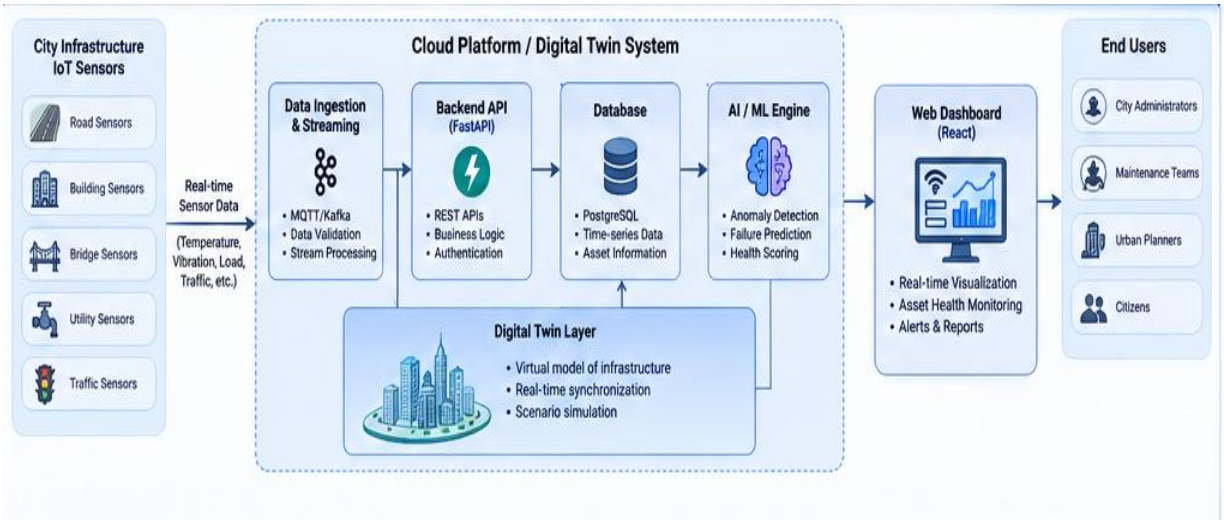
1. Create digital twins of city infrastructure.
2. Integrate real-time IoT data.
3. Predict infrastructure failures.
4. Monitor asset health continuously.
5. Support predictive maintenance.

1.4 Key Functionalities

The major functionalities of the platform include:

- Real-time infrastructure monitoring.
- Digital twin creation and visualization.

- IoT/sensor data integration.
- Asset health monitoring.
- Anomaly detection.
- Infrastructure failure prediction.



2. PROPOSED SOLUTION

The proposed platform consists of multiple software components working together to provide an integrated infrastructure monitoring system.IoT sensors or an IoT simulator generate infrastructure data such as temperature, vibration, pressure, structural conditions, traffic-related information, or other relevant parameters. The data is transmitted to the backend through an API or data-ingestion layer.The backend validates and processes the incoming data before storing it in the database. Historical information can be used by AI components to identify patterns and detect abnormal infrastructure conditions.The digital twin layer maintains a virtual representation of the physical asset. The current condition of each asset can be visualized through a dashboard.

2.1 System Workflow

The overall workflow is:

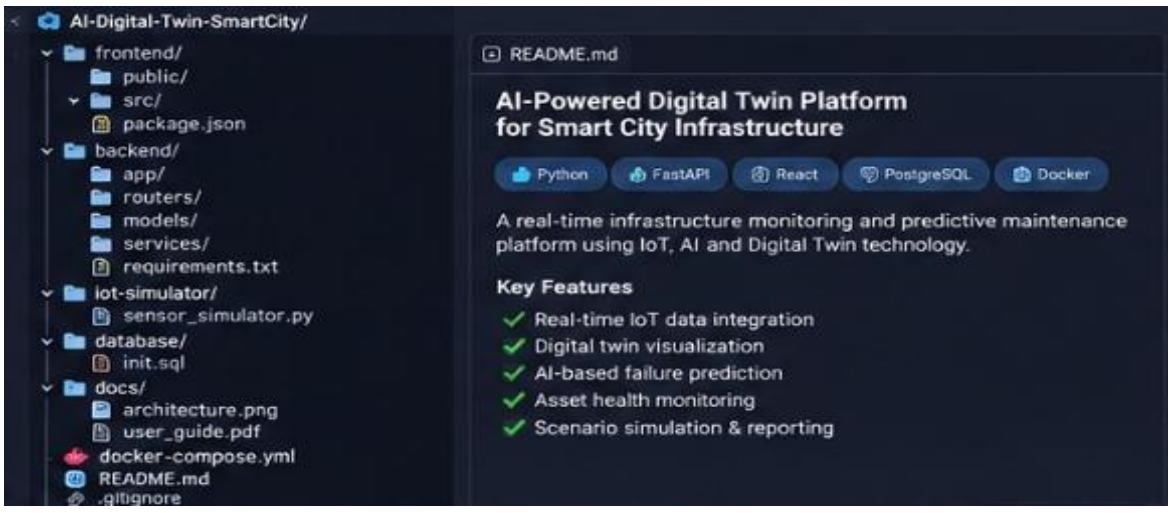
1. Sensors collect infrastructure data.
2. IoT data is transmitted to the platform.
3. Backend services receive and validate the data.
4. Data is stored in the database.
5. AI algorithms analyze the data.
6. Anomalies and possible failures are identified.
7. The digital twin is updated.

3. REPOSITORY ORGANIZATION

A well-organized repository improves readability, maintainability, collaboration, and future development. The proposed repository separates the frontend, backend, IoT simulation, and database components.

3.1 Repository Structure

AI-Digital-Twin/



3.2 Folder Responsibilities

frontend/

The frontend contains the user interface of the application. It provides dashboards, asset information, charts, maps, alerts, and digital twin visualizations.

backend/

The backend contains APIs and application logic. It receives sensor data, processes requests, communicates with the database, and provides information to the frontend.

iot-simulator/

The IoT simulator generates sensor information for testing and demonstration when physical sensors are unavailable.

database/

The database folder contains database initialization scripts and related configuration required for storing infrastructure and sensor information.

docker-compose.yml

Docker Compose can be used to coordinate multiple services and provide a reproducible development environment.

README.md

The README should contain project information, installation steps, dependencies, usage instructions, repository structure, and testing instructions.

3.3 Maintainability

The repository structure supports maintainability because each major responsibility is separated into an appropriate directory. Developers can work on the frontend, backend, IoT simulator, or database independently.Clear file names and consistent folder naming also make the repository easier to understand for new contributors.

4. CODE QUALITY REVIEW

The assessment requires evaluation of readability, modularity, coding standards, documentation, strengths, and areas for improvement.

4.1 Code Readability

Readable code is important because multiple developers may work on different components of the platform.

The following practices should be followed:

- Use meaningful variable and function names.
- Maintain consistent indentation.

- Use appropriate comments for complex logic.
- Avoid unnecessarily long functions.
- Organize related functionality into modules.
- Maintain consistent coding conventions.

For example, names such as get_asset_health() and predict_failure() are more meaningful than generic names such as function1().

4.2 Modularity

The platform follows a modular approach by separating:

- Frontend components.
- Backend APIs.
- Database operations.
- IoT data generation.
- AI processing.
- Digital twin visualization.

This separation allows individual components to be modified or extended without affecting the entire application.

4.3 Coding Standards

The following coding practices are recommended:

- Maintain consistent naming conventions.
- Validate incoming API data.
- Handle exceptions properly.
- Avoid duplicate code.
- Keep configuration separate from source code.

4.4 Strengths

The major strengths of the proposed implementation are:

1. Clear separation between frontend and backend.
2. Modular repository structure.
3. IoT integration capability.
4. AI-based predictive functionality.
5. Digital twin representation.

4.5 Areas for Improvement

The following improvements can increase code quality:

- Add automated unit testing.
- Add integration testing.
- Introduce centralized error handling.
- Add structured logging.
- Use static code analysis and linting.
- Improve API documentation.

5. VERSION CONTROL EVALUATION

Version control is an important part of collaborative software development. Git allows developers to maintain a history of changes, create branches, review modifications, and safely merge contributions.

The assessment specifically requires evaluation of commit history, branching strategy, commit messages, and repository maintenance practices.

5.1 Branching Strategy

A suitable branching structure for the project can be:

main

│

├── feature/frontend-dashboard

├── feature/backend-api

├── feature/iot-simulator

├── feature/ai-prediction

└── fix/api-validation

The main branch can contain the stable version of the project.

Feature branches can be used to develop individual functionalities without directly modifying the stable code.

5.2 Commit Messages

Commit messages should clearly explain the change made.

Examples:

Add infrastructure dashboard

Add sensor simulation module

Create asset health API

Implement anomaly detection

Fix telemetry validation

Update README installation steps

Meaningful commits make it easier to understand project development history.

5.3 Repository Maintenance

Good Git practices include:

- Avoid committing passwords or API keys.
- Maintain a proper .gitignore.
- Use meaningful commit messages.
- Use branches for major features.
- Review changes before merging.

5.4 Collaboration

Version control allows different team members to work simultaneously on different parts of the application.For example, one developer can work on the React dashboard while another develops FastAPI endpoints. Their changes can later be reviewed and merged into the main project.

6. CODE TESTING AND VALIDATION

Testing ensures that the application behaves according to its requirements. The assessment requires appropriate test cases, execution results, and screenshots as evidence.

6.1 Test Cases

6.2 Testing Workflow

Testing can be performed at different levels:

Unit Testing

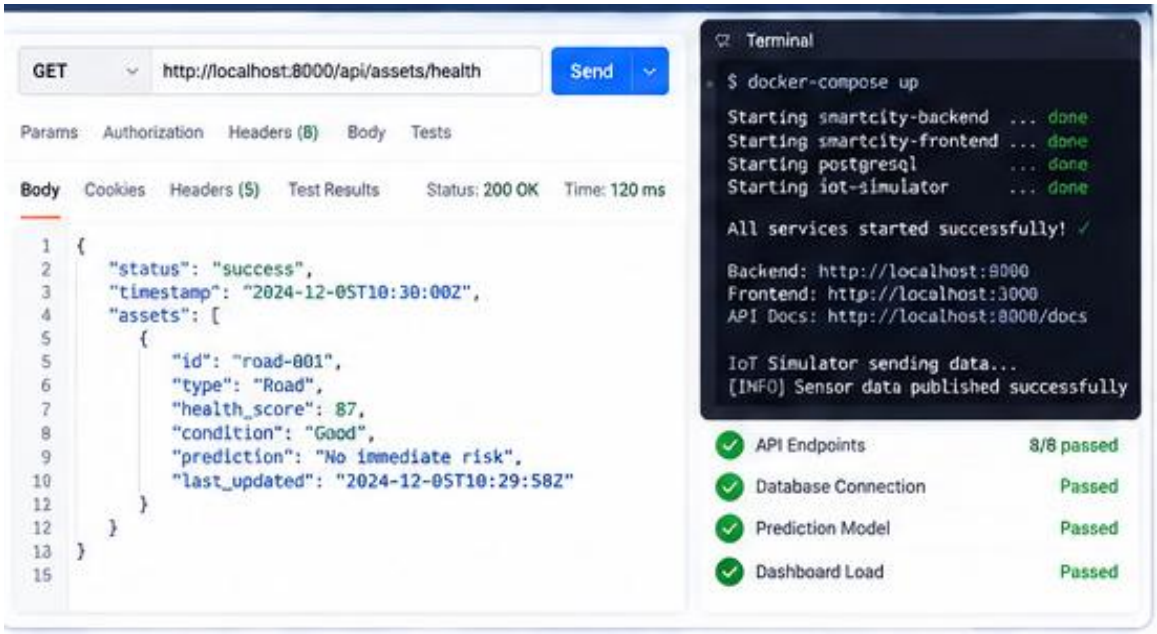
Individual functions such as data validation and prediction functions are tested independently.

API Testing

Backend endpoints are tested using valid and invalid inputs.

Integration Testing

The interaction between the frontend, backend, database, and IoT simulator is tested.



6.3 Evidence

The final assessment should include screenshots showing:

- Successful application execution.
- API request and response.
- Dashboard output.
- Alert/anomaly detection.
- Prediction result.
- Docker/service execution if applicable.

Actual execution results should be inserted from the implemented system rather than being assumed.

7. REPOSITORY DOCUMENTATION

The assessment requires evaluation of the README, installation instructions, usage guide, and dependencies.

7.1 README

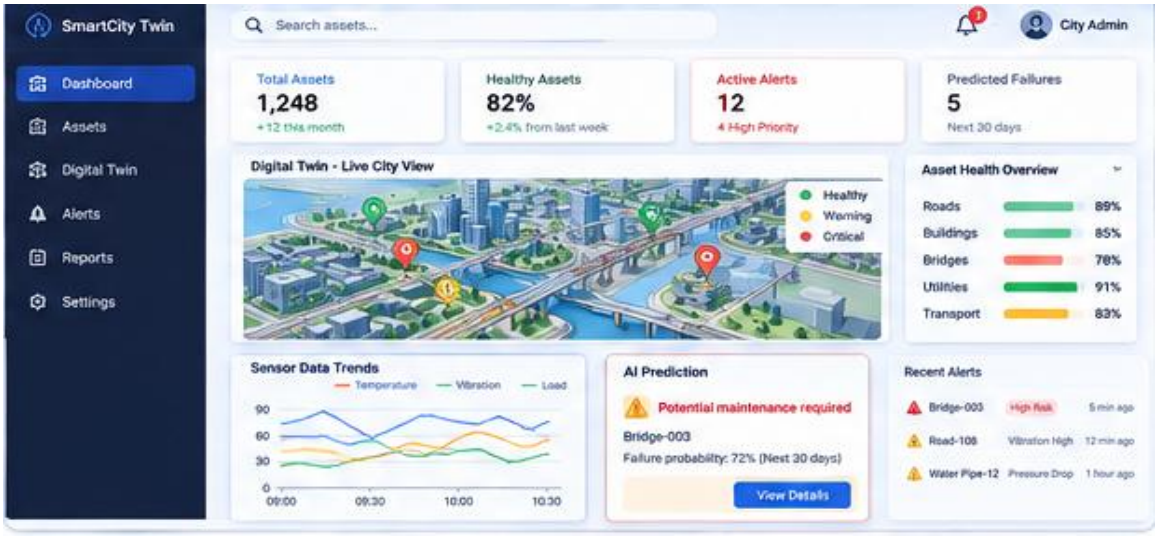
The README should contain:

1. Project title.
2. Project description.
3. Problem statement.
4. Objectives.
5. Features.

7.2 Installation Documentation

Installation instructions should clearly explain:

- Required software.
- Required dependencies.
- Environment variables.
- Database configuration.
- Backend setup.
- Frontend setup.



8. ARCHITECTURE AND QUALITY ATTRIBUTE ANALYSIS

The assessment also evaluates system requirements, software architecture, architecture diagram and component design, component interaction, quality attributes, and architectural justification.

8.1 Component Interaction

The major components interact as follows:

IoT Sensors → Data Ingestion → Backend → Database → AI Analysis → Digital Twin → Dashboard

The IoT layer provides data to the backend. The backend validates and processes the information. The database maintains current and historical information.The AI component analyzes the data to identify anomalies and predict potential failures.The digital twin combines asset information, current status, historical information, and analytical results.The frontend dashboard presents this information to city administrators.

8.2 Quality Attributes

Scalability

The modular architecture allows additional infrastructure assets and sensors to be added without redesigning the entire system.

Maintainability

Separating frontend, backend, database, and IoT components makes maintenance easier.

Reliability

Input validation, error handling, database persistence, and service monitoring can improve system reliability.

Performance

Efficient data processing and database access are important for real-time infrastructure monitoring.

Security

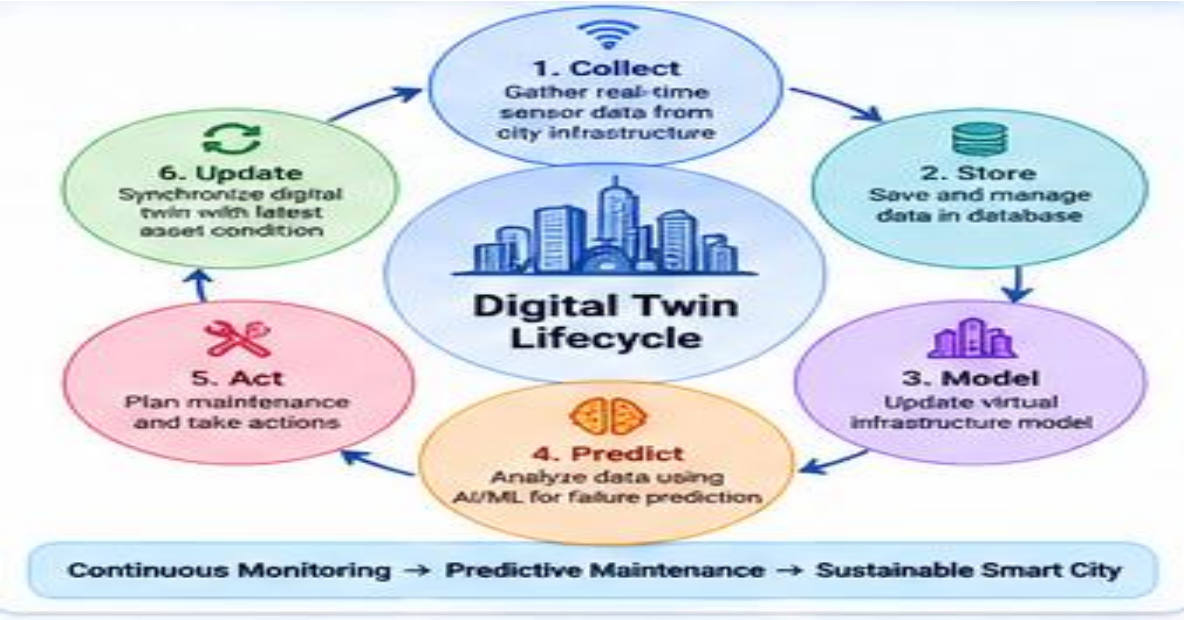
Sensitive configuration such as passwords and API keys should be stored using environment variables or secure secret-management mechanisms.

8.3 Digital Twin Lifecycle

The lifecycle can be represented as:

Collect → Store → Model → Predict → Act → Update

Sensor information is collected and stored. The digital model is updated based on the data. AI algorithms analyze the asset condition and predict possible failures. Maintenance actions can then be planned, after which the digital twin is updated with the latest asset condition.



9. CODE REVIEW FINDINGS, RECOMMENDATIONS AND CONCLUSION

9.1 Overall Findings

The AI-Powered Digital Twin Platform provides an appropriate solution for smart-city infrastructure monitoring. It combines IoT data, backend processing, database storage, AI-based analysis, digital twin representation, and dashboard visualization.The modular organization improves maintainability and allows different components to be developed independently.

9.2 Recommendations

The following improvements are recommended:

1. Implement comprehensive automated testing.
2. Maintain meaningful Git commit history.
3. Use feature branches for collaborative development.
4. Improve README and installation documentation.
5. Add centralized logging.
6. Add proper exception handling.

9.3 Expected Impact

The proposed platform can provide:

- Enhanced infrastructure monitoring.
- Reduced maintenance costs.
- Improved asset reliability.
- Better urban planning decisions.
- Predictive maintenance capabilities.
- Increased operational efficiency.

9.4 Conclusion

The AI-Powered Digital Twin Platform for Smart City Infrastructure provides a software-based approach for improving urban infrastructure management.By integrating IoT data, backend services, databases, AI analytics, and digital twin visualization, the platform can provide a unified view of infrastructure conditions.The proposed architecture supports modular development, maintainability, scalability, and collaborative software development. AI-based anomaly detection and failure prediction can further support predictive maintenance and help infrastructure managers make informed decisions.Overall, the project demonstrates how modern software engineering concepts can be applied to smart-city infrastructure management. Further improvements in automated testing, repository management, documentation, security, AI model validation, and GIS visualization can make the platform more reliable and suitable for future real-world deployment.